

Absolutely. This is a good **lab/exam exercise format** for teaching doubly linked lists. Students can be given the **node address and data values**, but they must draw the linked-list figure by determining the `prev` and `next` relationships.

Below are **10 problems with solutions**. I have deliberately separated the **student question** from the **solution** so you can give only the problem section to students.

Doubly Linked List — Figure Construction Problems

Node representation

Assume every node has three fields:

<code>prev</code>	<code>data</code>	<code>next</code>
Address of previous node	Data value	Address of next node

Special values:

- `NULL` = no previous/next node
- The **first node** has `prev = NULL`
- The **last node** has `next = NULL`

Part A — Problems for Students

Problem 1 — Create a Basic Doubly Linked List

The following nodes are stored in memory:

Node Address	Data
1000	10
1020	20
1040	30
1060	40

The `HEAD` points to node `1000`.

The nodes are arranged in the order:

1000 → 1020 → 1040 → 1060

Task

1. Draw the doubly linked list.
2. Show `prev` and `next` for every node.
3. Mark `HEAD` and `NULL`.
4. Write the complete contents of each node.

Solution

Address	prev	data	next
1000	NULL	10	1020
1020	1000	20	1040
1040	1020	30	1060
1060	1040	40	NULL

Figure:

HEAD

↓

NULL ← [10] ⇌ [20] ⇌ [30] ⇌ [40] → NULL
 1000 1020 1040 1060

Problem 2 — Construct from Given Addresses

Consider the following nodes:

Address	Data
2000	15
2050	25
2100	35
2150	45
2200	55

HEAD = 2000

The nodes are linked in this order:

2000 → 2050 → 2100 → 2150 → 2200

Task

Draw the doubly linked list and complete the following table.

Address	prev	Data	next
2000	?	15	?
2050	?	25	?
2100	?	35	?
2150	?	45	?
2200	?	55	?

Solution

Address	prev	Data	next
2000	NULL	15	2050
2050	2000	25	2100
2100	2050	35	2150
2150	2100	45	2200
2200	2150	55	NULL

Figure:



Problem 3 — Addresses Are Not in Sequential Order

The following nodes are stored at different memory locations.

Address	Data
3100	70
1500	20
4200	90
2750	50
6000	100

HEAD = 1500

The logical order is:

1500 → 2750 → 3100 → 4200 → 6000

Task

Students must:

1. Draw the doubly linked list.
2. Determine the `prev` and `next` addresses.
3. Do **not** arrange nodes according to numerical address order.

Solution

Address	prev	Data	next
1500	NULL	20	2750
2750	1500	50	3100
3100	2750	70	4200
4200	3100	90	6000
6000	4200	100	NULL

Figure:

```

HEAD
↓
NULL ← [20] ⇌ [50] ⇌ [70] ⇌ [90] ⇌ [100] → NULL
      1500   2750   3100   4200   6000

```

Important concept: Memory addresses do not determine the logical order of a linked list.

Problem 4 — Find the Missing Links

The following nodes are given:

Address	Data
5000	11
5030	22
5060	33
5090	44
5120	55

The following information is known:

- `HEAD = 5000`
- Node `5000` is the first node.
- Node `5120` is the last node.
- The logical order is:

5000 → 5060 → 5120 → 5030 → 5090

Task

Complete the table.

Address	prev	Data	next
5000	?	11	?
5030	?	22	?
5060	?	33	?
5090	?	44	?
5120	?	55	?

Solution

Address	prev	Data	next
5000	NULL	11	5060
5060	5000	33	5120
5120	5060	55	5030
5030	5120	22	5090
5090	5030	44	NULL

Figure:

HEAD

↓

NULL ← [11] ⇌ [33] ⇌ [55] ⇌ [22] ⇌ [44] → NULL
 5000 5060 5120 5030 5090

Problem 5 — Insert a New Node

Initially:

Address	Data
7000	10
7040	20
7080	40
7120	50

Initial logical order:

7000 → 7040 → 7080 → 7120

A new node is created at address 7160 with data 30.

The new node must be inserted **between 20 and 40**.

Task

1. Draw the list before insertion.
2. Draw the list after insertion.

- 3. Show the changed prev and next links.
- 4. Complete the final table.

Solution

Final logical order:

7000 → 7040 → 7160 → 7080 → 7120

Address	prev	Data	next
7000	NULL	10	7040
7040	7000	20	7160
7160	7040	30	7080
7080	7160	40	7120
7120	7080	50	NULL

Figure:

NULL ← [10] ⇌ [20] ⇌ [30] ⇌ [40] ⇌ [50] → NULL
7000 7040 7160 7080 7120

Problem 6 — Delete a Node

Consider:

Address	Data
8000	5
8040	15
8080	25
8120	35
8160	45

Logical order:

8000 → 8040 → 8080 → 8120 → 8160

Delete the node at address 8080.

Task

1. Draw the original list.
2. Remove node 8080 .
3. Update the prev and next links.
4. Draw the final list.

Solution

After deletion:

8000 → 8040 → 8120 → 8160

Address	prev	Data	next
8000	NULL	5	8040
8040	8000	15	8120
8120	8040	35	8160
8160	8120	45	NULL

Figure:

```
NULL ← [5] ⇌ [15] ⇌ [35] ⇌ [45] → NULL
      8000   8040   8120   8160
```

The links changed are:

```
8040.next = 8120
8120.prev = 8040
```

Problem 7 — Insert at the Beginning

The existing list is:

Address	Data
9000	30

Address	Data
9040	40
9080	50

Logical order:

9000 → 9040 → 9080

A new node is created:

Address	Data
8960	20

Insert this node at the **beginning**.

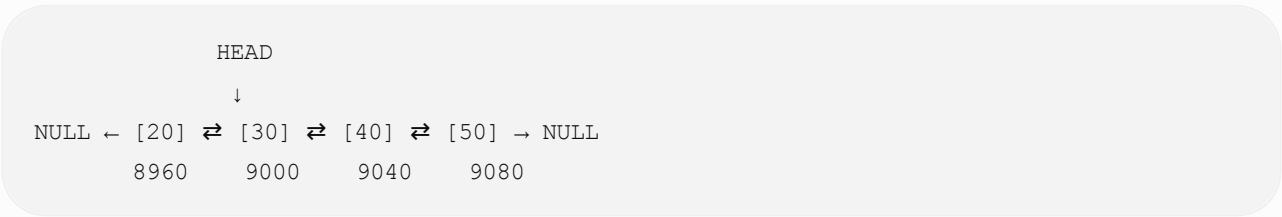
Solution

New logical order:

8960 → 9000 → 9040 → 9080

Address	prev	Data	next
8960	NULL	20	9000
9000	8960	30	9040
9040	9000	40	9080
9080	9040	50	NULL

Figure:



Problem 8 — Insert at the End

Given:

Address	Data
10000	100
10100	200
10200	300

Logical order:

10000 → 10100 → 10200

A new node:

Address	Data
10300	400

Insert it at the **end**.

Solution

Final order:

10000 → 10100 → 10200 → 10300

Address	prev	Data	next
10000	NULL	100	10100
10100	10000	200	10200
10200	10100	300	10300
10300	10200	400	NULL

Figure:

NULL ← [100] ⇌ [200] ⇌ [300] ⇌ [400] → NULL
 10000 10100 10200 10300

Problem 9 — Reverse Traversal

Consider the following doubly linked list:

Address	Data
1200	10
1350	20
1480	30
1650	40
1800	50

Logical order:

1200 → 1350 → 1480 → 1650 → 1800

Task

1. Draw the doubly linked list.
2. Identify the `TAIL`.
3. Starting from `TAIL`, perform reverse traversal.
4. Write the data values obtained during reverse traversal.

Solution

```

HEAD
↓
NULL ← [10] ⇌ [20] ⇌ [30] ⇌ [40] ⇌ [50] → NULL
        1200   1350   1480   1650   1800
                        ↑
                        TAIL

```

Reverse traversal:

50 → 40 → 30 → 20 → 10

Problem 10 — Challenge Problem

The following nodes exist in memory:

Address	Data
2100	12

Address	Data
2250	24
2400	36
2550	48
2700	60
2850	72

The linked-list order is:

2100 → 2400 → 2850 → 2250 → 2700 → 2550

HEAD = 2100

Tasks

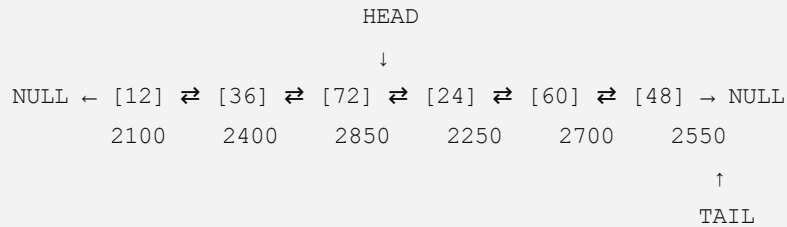
Students must:

1. Draw the complete doubly linked list.
2. Fill in `prev` and `next`.
3. Identify `HEAD` and `TAIL`.
4. Write the forward traversal.
5. Write the backward traversal.

Solution

Address	prev	Data	next
2100	NULL	12	2400
2400	2100	36	2850
2850	2400	72	2250
2250	2850	24	2700
2700	2250	60	2550
2550	2700	48	NULL

Figure:



Forward traversal:

12 → 36 → 72 → 24 → 60 → 48

Backward traversal:

48 → 60 → 24 → 72 → 36 → 12

Suggested Student Worksheet Format

For a practical class, you can give students a blank table like this:

Question

Given the following nodes:

Address	Data
4000	10
4500	20
5200	30
6100	40
7000	50

Logical order:

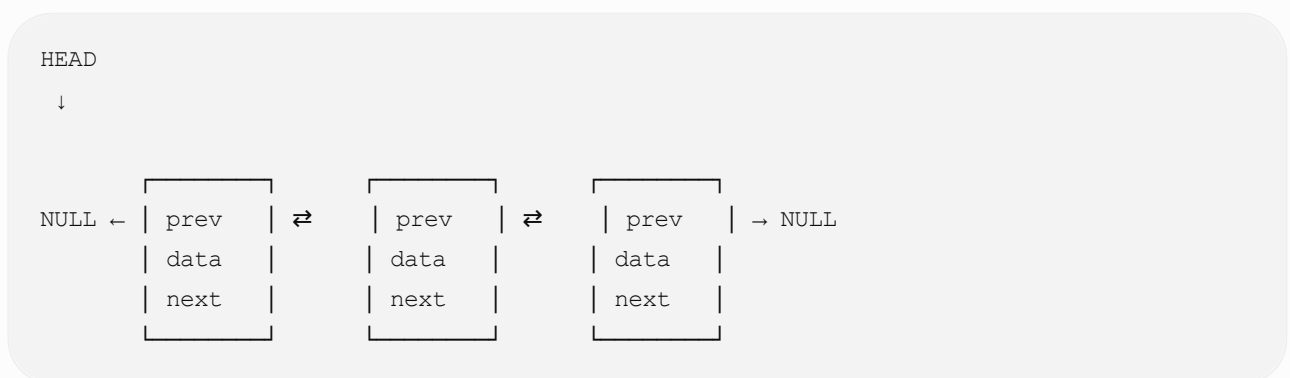
4000 → 5200 → 7000 → 4500 → 6100

HEAD = 4000

Student has to complete:

Address	prev	Data	next
4000	_____	10	_____
4500	_____	20	_____
5200	_____	30	_____
6100	_____	40	_____
7000	_____	50	_____

Draw the figure



Answer

Address	prev	Data	next
4000	NULL	10	5200
5200	4000	30	7000
7000	5200	50	4500
4500	7000	20	6100
6100	4500	40	NULL

A useful rule for students

Tell students to remember this simple rule:

For every pair of connected nodes:

```
Node A.next = Node B's address
```

For example:

```
[Address 1000] ⇌ [Address 2000]
```

```
1000.next = 2000
```

```
2000.prev = 1000
```

And at the two ends:

```
HEAD node.prev = NULL
```

```
TAIL node.next = NULL
```

This makes the exercises suitable for progressing from **basic figure construction** → **address-based linking** → **insertion** → **deletion** → **forward/reverse traversal**.